

Sprint 1 Requirements Artifacts

Overview:

When the user starts the game on the local host, they will be greeted with a login page to enter their username and password, and they will be able to login and see their saved information. If they don't have an account yet, then they can make an account.

Functional Requirements:

User Story: As a user I want to be able to login to the game

Login Functionality: Dustin

FR1.1: The user will have a box to enter their username and password

FR1.2: If the user does not have an account, they can click the sign-up button to create one

FR1.2a. In the signup screen, the user will create a user account to be able to enter the game.

FR1.2b: The user will click "confirm sign up" to be registered and be entered into the dojo

FR1.3: From the login screen, when the user clicks the "Enter Dojo" button, they will either be signed in or be told that their credentials are incorrect.

FR1.4: If no account was found, a prompt message whether a credential is invalid, or a text box is empty will appear.

Overview:

When the user enters a game then the battle screen will appear with their deck, and they can play the game

Functional Requirements:

User Story: As a user I want to be able to battle against other people and decks in a different screen than the home page

Battle Screen: Colin

FR2.1: Create a new "battle page" for the user to see when they join a battle against someone

FR2.2: Display the users deck info on the page

FR2.3: User can play the game against an ai

FR3.4: When either the user or ai wins it will show a result screen and allow the user to return to the home page

Sprint 1 Requirements Artifacts

Overview:

When a user creates an account, they want to know their information about their progress and status in the game to be saved and that no one else is going to be able to login and play the game as them. They also want to know that if they want, they can delete their account.

Functional Requirements:

User Story: As a user I want to know that the data I'm seeing is relevant to my account, that no one else is going to be able to edit my data.

User Services CRUD: Hannah

FR2: Create file structure so that all the routes and controllers are stored together.

FR2: Create files user services routes and user services controllers.

FR2: Write routes for get, post, put and delete https requests that link to controller functions

FR3: write controller functions that handle business logic for get, post, put and delete operations that will eventually update the database and that do pass back a https response.

FR4: write a function that hashes the password securely so that the database isn't storing plain text passwords, then update post so that it leverages that and get so that it authorizes the user correctly comparing hashes.

FR5: Write Jest tests to test the routes and controllers

Overview:

When a user is logging in, playing the game, or changing the cards in their deck the client makes a request to a http server to update their information or make a request for info about a user.

Functional Requirements:

User Story: As a user I want my info to be updated easily and quickly when making changes.

Database interface: Ryland

FR1.1. create web server to handle incoming requests

FR1.2 .create routing functions to handle what kind of request is being made

Sprint 1 Requirements Artifacts

FR1.3. create query functions that are called by the routing functions to carry out the correct sql queries to the database

FR1.4. create table schemas for the database to hold user data and overall game state.

Overview:

A user is able to play cards within a battle that follows a set of rules per turn, allowing them to build a unique strategy and deck to their liking.

Functional Requirements:

User Story: As a user, I want to enter into battles using the deck that I have built.

Game Structure: Riley

FR1.1. Maintain card type hierarchy and consistent battle mechanics through the course of a match

FR1.2. Create a parsing system for consistently and accurately applying card effects in alignment with what the card says it will do.

FR1.3. Use deck pulled from player profile, shuffling the order so that it cannot be predicted.

FR1.4. Procedurally generate cards with consistent IDs.

Overview

The Card Deck System handles the generation and management of card sets used during random encounters in Dodo Dojo. Each encounter uses a deck of cards corresponding to the elemental type of the Dodo being challenged (e.g., Fire, Ice, Earth, etc.). This ensures each enemy has a balanced but themed deck, allowing for fair and varied gameplay. The deck system contributes to replayability and supports deck progression and card drops for the player.

Functional Requirements

User Story: As a user, I want each random encounter to use a unique and balanced deck of cards based on the opponent's element, so that every battle feels fresh and challenging.

Random Encounter Card Decks – Jacob Richards

FR1.1: The system shall initialize enemy decks by drawing cards from the total available card pool, with preference for the enemy's elemental type.

FR1.2: The system shall ensure decks contain a maximum of 30 cards, with at least 50% belonging to the enemy's type.

Sprint 1 Requirements Artifacts

FR1.3: The system shall generate decks dynamically at the start of each encounter to promote variety and fairness.

FR1.4: The system shall allow cards to drop as rewards (25% probability) after a loss, allowing players to expand their own deck.

FR1.5: The system shall support future combination features where players can merge lower-level cards into stronger versions of the same type.